

“고등부 1번. 점프” 문제 풀이

작성자: 이종서

부분문제 1

처음 방문하는 정점이 1이고 마지막으로 방문하는 정점이 N 일 때 가능한 정점의 방문 순서는 총 $(N-2)!$ 가지 있다. 이 방문 순서를 전부 순회하며 간선을 지난 횟수가 입력과 같아지는 것을 찾아 문제를 해결할 수 있다. 시간 복잡도는 $O(N^2 \times (N-2)!) = O(N!)$ 이다.

부분문제 2

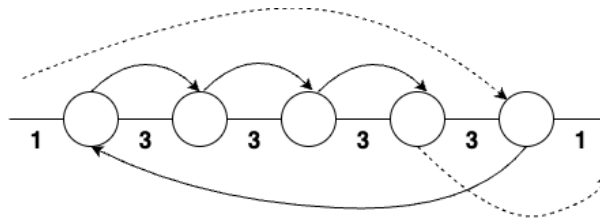
간선을 지난 횟수를 나타내는 수열 c 에서는 다음과 같이 연속된 1과 연속된 3이 반복되어 나타나게 된다.

$$c = (\underbrace{1, 1, \dots, 1}_{\text{연속된 1}}, \underbrace{3, 3, \dots, 3}_{\text{연속된 3}}, \dots, \underbrace{1, 1, \dots, 1}_{\text{연속된 1}})$$

$c_i = 1$ 인 간선들은 정방향으로 한 번 지나야 하고, $c_i = 3$ 인 간선들은 정방향으로 두 번, 역방향으로 한 번 지나야 함을 관찰할 수 있다.

이를 기반으로 그래프의 간선들을 $c_i = 1$ 인 연속된 간선의 구간과 $c_i = 3$ 인 연속된 간선의 구간으로 나누어 생각할 수 있다. 구간을 앞에서부터 순회하며,

- $c_i = 1$ 구간에서는 간선들을 한 번만 지나면 되므로, 대응되는 정점들을 순방향(정점 번호가 증가하는 순서)으로 하나씩 방문한다.
- $c_i = 3$ 구간에서는 간선들을 역방향으로도 한 번 지나야 하므로, 구간의 마지막 정점을 먼저 방문한 후, 시작 정점으로 돌아와 나머지 정점을 차례로 하나씩 방문한다.



이를 구현하면 $O(N)$ 시간 복잡도에 해결할 수 있다. 구현시 두 구간의 경계에 걸쳐있는 정점에 대한 처리에 유의해야 한다.

부분문제 3

연속한 동일한 c 값을 갖는 간선들은 한 번에 연달아 통과(방문)해도 무방하다. 따라서

$$c_1 < c_2 < \dots < c_M > c_{M+1} > \dots > c_{N-1}$$

와 같이 입력이 강증가 후 강감소 형태라고 가정할 수 있다. 이때는 항상

$$|c_i - c_{i+1}| = 2$$

가 성립한다. 즉, c 수열이

$$(1, 3, \dots, 2k-3, 2k-1, 2k-3, \dots, 3, 1)$$

끝이면 충분하다.

이 경우의 방문 순서는

$$p = (1, 2k-1, 2, 2k-2, 3, 2k-3, \dots, k, 2k)$$

로 잡으면 모든 간선을 정확히 요구 횟수만큼 통과하면서 정점 1에서 $N = 2k$ 까지 도달할 수 있다.

이를 구현하면 $O(N)$ 시간에 해결할 수 있다.

부분문제 4

어떤 점프가 더 작은 번호의 정점에서 더 큰 번호의 정점으로 이동한다면 ‘순방향’, 그렇지 않다면 ‘역방향’이라고 부르자. 2 이상 $N-1$ 이하의 정수 i 에 대해 정점 i 의 앞/뒤 간선을 지난 횟수의 차이를 $d_i := c_i - c_{i-1}$ 로 나타내자. 만약, $c_1 = c_{N-1} = 1$ 이고, d_i 조건을 만족하는 점프 방법을 찾는다면, 문제를 해결할 수 있다.

먼저, 다음 사실을 관찰할 수 있다:

- $d_i \in \{+2, 0, -2\}$
- $d_i = +2$ 인 정점 i 는 직전/직후 방문 정점의 번호가 모두 i 보다 크다. 다시 말해, $p_j = i$ 라면,

$$p_{j-1}, p_{j+1} > i$$

- $d_i = -2$ 인 정점 i 는 직전/직후 방문 정점의 번호가 모두 i 보다 작다. 다시 말해, $p_j = i$ 라면,

$$p_{j-1}, p_{j+1} < i$$

- $d_i = 0$ 인 정점 i 는 직전/직후 방문 정점들의 i 와의 대소 관계가 서로 다르다. 다시 말해, $p_j = i$ 라면,

$$\min(p_{j-1}, p_{j+1}) < i < \max(p_{j-1}, p_{j+1})$$

두 집합 $S := \{i \mid d_i = -2\}$ 와 $E := \{i \mid d_i = +2\}$ 를 정의하자. $S := \{s_1, s_2, \dots\}$, $E := \{e_1, e_2, \dots\}$ 와 같이 $s_1 \leq s_2 \leq \dots$ 이고 $e_1 \leq e_2 \leq \dots$ 이게 각 원소에 크기 순으로 번호를 붙이자. 이 때, $c_i - c_1 = \sum_{t=1}^{i-1} d_t$ 로 부터 다음 두 사실을 알 수 있다:

- $c_{n-1} = 1$ 이므로, $\sum d_t = 0$ 이 되어 $|S| = |E|$.
- 모든 i 에 대해 $c_i \geq 1$ 이므로, 항상 $\sum_{t=1}^{i-1} d_t \geq 0$. 고로 모든 i 에 대해 $s_i > e_i$.

E 의 각 정점들은 순방향 점프로 들어와서 역방향 점프로 나가야 하며, S 의 각 정점들은 역방향 점프로 들어와서 순방향 점프로 나가야 한다. 그리고 위에서 관찰한 사실에 의해 s_i 에서 출발해 e_i 에 도착하는 점프는 역방향 점프이다.

따라서, 다음과 같이 방문 순서 p 를 구성할 수 있다.

1. $p := (1)$

2. 각 $i = 1, 2, \dots, N - 1$ 에 대해:

- 어떤 j 가 존재해 $s_j = p_i$ 라면, $p_{i+1} := e_j$
- 아니라면, $p_{i+1} := \min \left\{ x \in \{1, 2, \dots, n\} \mid x \notin E \wedge x \notin \bigcup_{j \leq i} p_j \right\}$

위 방법으로 구한 방문 순서가 실제로 d_i 조건을 만족함은 쉽게 확인할 수 있다. 시간 복잡도는 $O(N^2)$ 이다.

부분문제 5

부분문제 4의 풀이에서 $p_{i+1} := \min \left\{ x \in \{1, 2, \dots, n\} \mid x \notin E \wedge x \notin \bigcup_{j \leq i} p_j \right\}$ 를 계산하는 과정에서 최대 $O(N)$ 시간이 걸려, 총 $O(N^2)$ 시간이 소요되었다.

이를 $\{1, 2, \dots, n\} \setminus E$ 을 미리 큐에 넣어 두고 차례로 꺼내는 방식으로 구현하면 전체 $O(N)$ 시간에 동작하게 된다.

“고등부 2번. 새로운 인연” 문제 풀이

작성자: 박상훈

부분문제 1

L_i 번 의자에 앉은 남자와 R_j 번 의자에 앉은 여자가 새로운 커플이 되는 것을 L_i 와 R_j 가 매칭된다고 하자. 각 L_i 에 매칭되는 R_j 는 $R_1, R_2, \dots, R_N$ 중 하나고, 모두 다르기 때문에 확인해야 할 경우의 수는 $N!$ 가지이다. 각 L_i 에 매칭되는 R_j 를 고정하고 나면 $i \neq j, L_i < R_j$ 조건만 확인하면 되고 이는 시간복잡도 $O(N)$ 에 확인할 수 있다. 따라서, 시간복잡도 $O(N \times N!)$ 에 문제를 해결할 수 있다.

부분문제 2

앞으로 $R_1 < R_2 < \dots < R_N$ 을 만족하도록 정렬해 놓고 생각한다. $R_1, R_2, \dots, R_N$ 순서로 매칭되는 L_i 를 골라준다. 현재까지 매칭된 L_i 의 집합을 S 라고 하고, 그 상태에 도달할 수 있는 경우의 수를 $dp[S]$ 로 정의한다. R_j 와 매칭되는 L_i 는 $L_i \notin S, L_i < R_j, i \neq j$ 를 만족해야 하고, 매칭하고 나면 S 는 $S \cup \{L_i\}$ 로 바뀌게 된다. R_j 와 매칭될 점을 고르기 직전에는 $|S| = j - 1$ 이므로, 각 S 에 대해 정확히 하나의 R_j 에서 $O(N)$ 번의 dp 전이가 발생한다. S 는 길이 N 의 비트마스크로 표현 가능하므로, 가능한 S 의 가짓수는 $O(2^N)$ 이다. 따라서, 시간복잡도 $O(N \times 2^N)$ 에 문제를 해결할 수 있다.

포함과 배제의 원리를 적용한 풀이도 있다. 이에 대해서는 부분문제 5의 풀이에 적어두었다.

부분문제 3

(L_i, R_i) 를 i 라는 정점 번호를 가진 정점으로 두고, $L_i < L_j < R_j < R_i$ 를 만족하는 (i, j) 에 대해, $L_i < L_k < L_j < R_j < R_k < R_i$ 를 만족하는 k 가 존재하지 않는다면 i 번 정점과 j 번 정점을 연결하는 간선을 추가하여 만든 그래프를 생각하자. 문제 조건에 의해 각 컴포넌트는 트리이고, 부분문제 3에서는 각 컴포넌트가 일직선임을 알 수 있다.

$R_1, R_2, \dots, R_N$ 순서로 보면서 R_1 이 속한 컴포넌트가 끝나는 지점을 생각하자. 이때, 매칭되지 않은 R_j 가 존재한다면 그러한 R_j 는 끝까지 매칭될 수 없다. 따라서, R_1 이 속한 컴포넌트에 있는 L_i 와 R_j 는 그 컴포넌트 내부에서만 매칭이 발생한다. 이러한 논리를 통해, 각 컴포넌트는 외부로 나가는 매칭이 존재하지 않음을 수학적 귀납법으로 증명할 수 있다. 따라서, 각 컴포넌트에 대해 독립적으로 문제를 해결한 후, 각각의 답을 모두 곱하면 전체 문제의 답이 된다.

컴포넌트가 일직선인 경우, 모든 i 에 대해 $P_i \neq i$ 를 만족하는 순열의 개수를 세는 문제와 동치다. 이는 교란순열이라는 이름으로 잘 알려진 문제이며, 시간복잡도 $O(N)$ 에 계산할 수 있다. 순열의 길이가 n 이고 $P_n = x$ 라고 하자. x 로 가능한 값은 $n - 1$ 가지이다. $P_x = n$ 인 경우, 길이 $n - 2$ 일 때의 문제와 동일한 상황이 된다. 그렇지 않은 경우, $P_y = n$ 인 $x \neq y$ 가 존재한다. P_n 을 지우고 P_y 를 x 로 바꿔서 만든 새로운 순열을 Q 라고 하면, Q 는 길이 $n - 1$ 의 교란순열이 되고, 이는 P 와 일대일 대응이다. 따라서, 길이 n 의 교란순열의 개수를 D_n 이라고 하면 $D_n = (n - 1)(D_{n-1} + D_{n-2})$ 가 성립한다. 따라서, 시간복잡도 $O(N)$ 에 부분문제 3을 해결할 수 있다.

부분문제 4

정점 i 와 j 를 연결하는 간선에 대해, $L_i < L_j < R_j < L_i$ 를 만족하면 i 를 j 의 부모라고 하자. 각 컴포넌트에 대해 부모가 없는 정점은 유일하게 존재하고, 이를 트리의 루트로 생각해 줄 수 있다. 트리의 루트가

정의되었기 때문에 서브트리, 조상 등도 잘 정의할 수 있다.

$dp[s][j]$ 를 s 번 정점의 서브트리만 고려했을 때, 매칭되지 않은 남자와 여자가 각각 j 명인 경우의 수라고 정의하자. dfs를 돌면서 dp 를 계산한다. s 번 정점의 자식을 $v_1, v_2, \dots, v_k$ 라고 하자. (단, $R_{v_1} < R_{v_2} < \dots < R_{v_k}$) $dp2[v_i][j]$ 를 $v_1, v_2, \dots, v_i$ 번 정점의 서브트리만 고려했을 때, 매칭되지 않은 남자와 여자가 각각 j 명인 경우의 수라고 정의하자. $dp2[v_{i+1}]$ 을 계산하기 위해서는 $dp2[v_i]$ 와 $dp[v_{i+1}]$ 을 합쳐줘야 한다. 이때, $v_1, v_2, \dots, v_i$ 번 정점의 서브트리에서 매칭되지 않은 남자 중 z 명을 고르고, v_{i+1} 번 정점의 서브트리에서 매칭되지 않은 여자 중 z 명을 골라서 매칭시켜 줄 수 있다. 이를 식으로 표현하면 $dp2[v_{i+1}][x+y-z] \leftarrow dp2[v_i][x] \times dp[v_{i+1}][y] \times \binom{x}{z} \binom{y}{z} \times z!$ 이다. ($a \leftarrow b$ 는 변수 a 에 b 를 더해준다는 의미다.)

$dp2[v_k]$ 를 $dp[s]$ 로 전이해 주기 위해, 현재 정점에 해당하는 L_i 와 R_i 를 매칭해줘야 한다. L_i 와 R_i 를 둘 다 매칭해 주지 않는 경우는 $dp[s][j+1] \leftarrow dp2[v_k][j]$ 이다. L_i 와 R_i 중 하나만 매칭해 줄 경우, L_i 와 R_i 중 선택하는 경우의 수 2가지, 매칭되는 사람의 경우의 수 j 가지를 곱해서 총 $2j$ 가지의 경우의 수가 생긴다. 즉, $dp[s][j] \leftarrow dp2[v_k][j] \times 2j$ 이다. L_i 와 R_i 가 둘 다 매칭될 경우, 매칭되는 사람을 고르는 경우의 수가 각각 j 가지이므로 이를 곱해서 총 j^2 가지 경우의 수가 생긴다. 즉, $dp[s][j-1] \leftarrow dp2[v_k][j] \times j^2$ 이다.

각 컴포넌트의 루트를 $r_1, r_2, \dots, r_c$ 라고 하면 답은 $dp[r_1][0] \times dp[r_2][0] \times \dots \times dp[r_c][0]$ 이다.

구현에 따라 시간복잡도는 $O(N^3)$ 또는 $O(N^4)$ 이다.

부분문제 5

포함과 배제의 원리로 생각한다. $S = \{i_1, i_2, \dots, i_k\}$ 에 대해, $i \in S$ 면 L_i 와 R_i 를 매칭해 주고, $i \notin S$ 면 L_i 와 R_i 를 매칭하면 안 된다는 조건을 무시한 채로 매칭해 준다. 이러한 경우의 수를 $f(S)$ 라고 하면, 포함과 배제의 원리에 의해 답은 $\sum (-1)^{|S|} f(S)$ 이다. 이를 직접 구현하면 시간복잡도가 $O(N \times 2^N)$ 이 되어 부분문제 1과 2를 해결할 수 있다.

고정된 S 에 대해, $R_1, R_2, \dots, R_N$ 순서로 보면서 R_i 를 매칭해주는 상황을 생각해 보자. $i \in S$ 면, $a_i = -1$ 로 정의하고, R_i 를 L_i 와 매칭해준다. $i \notin S$ 면, a_i 를 R_i 가 매칭될 수 있는 L_j 의 개수로 정의하고, 그 중 아무거나 골라서 매칭해 준다. 이때, $a_1 a_2 \dots a_N = (-1)^{|S|} f(S)$ 임을 알 수 있다. 한편, i 번 정점의 조상을 $p_1, p_2, \dots, p_d$ 라고 할 때, $i \notin S$ 면 a_i 는 $p_j \notin S$ 인 j 의 개수에 1을 더한 값과 동일하다. 이 사실을 바탕으로 dp 를 한다.

$dp[s][j]$ 를 s 의 조상 중 S 의 원소가 아닌 것이 $j-1$ 개일 때, s 의 서브트리에 속한 a_i 의 곱의 합으로 정의한다. 즉, $a_s = j$ 이거나 $a_s = -1$ 만 가능한 상황이다. $a_s = j$ 인 경우, 자식으로 갔을 때 j 값은 1 증가한다. 한편, $a_s = -1$ 인 경우, 자식으로 갔을 때 j 값은 그대로다. 이를 각각 식으로 적으면 $dp[s][j] \leftarrow (dp[v_1][j+1] \times \dots \times dp[v_k][j+1]) \times j$, $dp[s][j] \leftarrow (dp[v_1][j] \times \dots \times dp[v_k][j]) \times (-1)$ 이다.

답은 $dp[r_1][1] \times dp[r_2][1] \times \dots \times dp[r_c][1]$ 이다. 시간복잡도는 $O(N^2)$ 이다.

“중등부 4번 / 고등부 3번. 수열과 쿼리” 문제 풀이

작성자: 구재현

부분문제 1

쿼리가 주어졌을 때, i 를 고정하고 j 를 증가시키면서 모든 연속 구간을 열거하고 각각의 합을 계산할 수 있다. 이 중 최댓값을 출력하면 된다. 시간 복잡도는 $O(QN^2)$ 이다.

부분문제 2

배열 $S_i = \sum_{j=0}^{i-1} A_j$ 를 정의하자. 이는 점화식 $S_{i+1} = S_i + A_i - X$ 를 사용하여 $O(N)$ 에 계산할 수 있다.

쿼리 X 가 주어졌을 때, 문제의 정답은 $\max_{0 \leq j < i \leq N} (S_i + iX - S_j - jX) = \max_{1 \leq i \leq N} (S_i + iX - (\min_{0 \leq j \leq i-1} S_j + jX))$ 이다. 배열 $D_i = \min_{j \leq i} (S_j + jX)$ 를 정의하자. 이는 점화식 $D_{i+1} = \min(D_i, S_{i+1})$ 를 사용하여 $O(N)$ 에 계산할 수 있다. 문제의 정답은 $\max_{1 \leq i \leq N} (S_i + iX - D_{i-1})$ 이고, $O(N)$ 에 계산할 수 있다. 시간 복잡도는 $O(QN)$ 이다.

부분문제 3

M_l 을 길이 l 의 부분 배열 중 합의 최댓값이라고 정의하자. 배열 M 은 $O(N^2)$ 시간에 계산할 수 있다. 이제 문제의 정답은 $\max_{l=1}^N (M_l + lX)$ 이다. 이를 단순히 계산하면 쿼리당 $O(N)$ 시간이 걸리니, 최적화하자.

2차원 평면 상에 N 개의 점 $(x_l, y_l) = (l, M_l)$ 을 그렸을 때, 쿼리 P 에 대해 우리는 $x_l \cdot P + y_l$ 의 최댓값을 계산하고 싶다 (입력에는 X 로 주어지지만 이제부터 혼동을 막기 위해 P 로 표현한다). N 개의 점들로 볼록 껍질 (convex hull) 을 구했을 때, 최댓값이 얻어지는 점은 볼록 껍질의 위쪽에 기울기 $-P$ 의 직선을 그었을 때의 접점임을 알 수 있다. 이러한 접점은, 볼록 껍질의 위 부분 (upper convex hull) 을 구하고, 각 직선마다 이분 탐색을 사용하여 $O(\log N)$ 시간에 찾을 수 있다. 시간 복잡도는 $O(N^2 + Q \log N)$ 이다.

부분문제 6

배열 M 을 빠르게 구하는 것은 어렵기 때문에, 다른 접근이 필요하다.

부분문제 3의 알고리즘과 동치이지만 약간 더 비효율적인, 다음과 같은 알고리즘을 생각하자: 2차원 평면 상에 $N(N+1)/2$ 개의 점 $\{(i-j, S_i - S_j)\}_{0 \leq j < i \leq N}$ 이 주어졌을 때, 쿼리 P 에 대해 $x_l \cdot P + y_l$ 의 최댓값이 문제의 정답이 된다. 모든 $N(N+1)/2$ 개의 점들을 가지고 upper convex hull을 구한 이후, 부분문제 3과 똑같은 이분 탐색을 사용하면 각 쿼리를 $O(\log N)$ 시간에 계산할 수 있다. 하지만, 점의 개수가 너무 많기 때문에, 컨벡스 헐을 구하는 것이 쉽지 않아 보인다.

이 문제를 분할 정복으로 해결하자. $f(L, R)$ 을, 점 집합 $\{(i-j, S_i - S_j)\}_{L \leq j < i \leq R}$ 의 upper convex hull을 반환하는 함수로 정의하자. $M = (L+R)/2$ 로 정의하면, $f(L, M), f(M+1, R)$ 을 호출하여 $i \leq M, j \geq L+1$ 인 경우의 컨벡스 헐의 후보를 모두 얻을 수 있다. 따라서, 점 집합 $\{(i-j, S_i - S_j)\}_{L \leq j \leq M < i \leq R}$ 의 upper convex hull을 계산하고, 재귀적으로 계산한 컨벡스 헐의 후보랑 합쳐준 다음에, 해당 점들로 upper convex hull을 다시 계산한 후 반환하면 된다.

어떠한 점 $(i-j, S_i - S_j)$ 가 upper convex hull에 있다는 것은, 기울기 p 가 존재해서 $(i-j)p + S_i - S_j$ 의 최댓값이 해당 점에서 유일하게 얻어진다는 것과 동치이다. (다시 말해, 모든 다른 점들은 $(i-j)p + S_i - S_j$ 의 값이 해당 점보다 작다). 이를 위해서는 $i \in [M+1, R]$ 에서 $S_i + ip$ 가 최대여야 하고, $j \in [L, M]$ 에서 $S_j + jp$ 가 최소여야 한다. 이는, $M+1 \leq i \leq R$ 에 있는 점 (i, S_i) 중 upper convex hull에 있는 점들만

고려해도 되고, $L \leq j \leq M$ 에 있는 점 (j, S_j) 중 lower convex hull에 있는 점들만 고려해도 된다는 것과 동치이다.

$\{(i, S_i)\}_{i=M+1}^R$ 에 있는 점들로 upper convex hull을 구하게 되면, 컨벡스 헐 상에서 인접한 점들을 사용하여 각 점이 최댓값을 이루는 p 의 범위를 계산할 수 있다. 같은 방식으로, $\{(j, S_j)\}_{j=L}^M$ 에 있는 점들로 lower convex hull을 구하게 되면, 각 점이 최솟값을 이루는 p 의 범위를 계산할 수 있다. 위의 논리로 이 p 의 범위가 겹치는 (i, j) 들의 쌍들만 계산하면 되고, 이는 머지 소트에서 병합하는 과정과 유사한, 아주 짧은 코드로 해결할 수 있다. (이러한 계산을 Minkowski Sum이라고 부른다.)

분할 정복 알고리즘은 컨벡스 헐을 구하는 과정에서 정렬을 필요로 하기 때문에 $T(N) = 2T(N/2) + O(N \log N) = O(N \log^2 N)$ 시간을 필요로 한다. 고로 시간 복잡도는 $O(N \log^2 N + Q \log N)$ 이다.

부분문제 7

위 분할 정복을 $O(N \log N)$ 에 구현하면 된다. 먼저, $\{(i, S_i)\}_{i=M+1}^R$ 에 있는 점들로 upper convex hull을 구할 때, 자연스럽게 각 점을 i 가 증가하는 순서대로 본 후 Monotone Chain을 사용하면 $O(N)$ 시간에 upper convex hull을 계산할 수 있다. lower convex hull도 동일하다. 또한, $f(L, R)$ 이 정확히 upper convex hull을 반환하게끔 구현할 필요가 없다: 병합 과정에서 나온 점들을 아무 순서 없이 모은 후 마지막에 한번 upper convex hull을 계산해 주면 된다. 최종적으로, 쿼리 처리 직전에 upper convex hull을 계산해 줄 때, 각 점의 x 좌표가 $[1, N]$ 범위이기 때문에, 단순히 각 x 좌표에 대한 y 좌표 최댓값을 배열에 기록해 주는 방식으로 정렬을 대체할 수 있다. 시간 복잡도는 $O((N + Q) \log N)$ 이다.

Remark 1. 입력 제한이 커서, 컨벡스 헐 등을 CCW를 사용하여 계산하였을 때 long long 범위를 초과하는 문제가 발생할 수 있다. 이 문제는 네 가지 서로 다른 방법으로 해결할 수 있다 (편리성 증가하는 순으로 나열). 모범 코드는 두 번째 방법으로 구현되었다.

- CCW 대신 기울기를 비교하는 함수를 사용하자. 쿼리 값이 모두 정수이기 때문에, 기울기를 정확한 실수가 아니라 정수 몫을 취한 값으로 부정확하게 비교하여도 문제가 없다.
- CCW의 값을 double 자료형으로 계산한 후, 절댓값이 10^{18} 이하일 경우 long long 으로 (오버플로우가 나도록) 다시 정확히 계산.
- CCW의 값을 long double 자료형으로 계산.
- CCW의 값을 __int128 자료형으로 계산: 이번 대회에 사용된 Biko의 채점 환경은 이를 지원한다.

Remark 2. 이 문제에 대해 기존에 알려진 풀이는 Sakai의 2024년 논문으로 [Sak24], $O(N \log^2 N)$ 의 전처리 시간이 걸린다. 해설의 풀이는 동일한 공간 복잡도와 쿼리 시간을 유지하며, 전처리 시간을 $O(N \log N)$ 으로 개선한다. 또한, 해설의 풀이는 논문의 접근과 비교했을 때 훨씬 단순하다.

References

- [Sak24] Yoshifumi Sakai. A data structure for the maximum-sum segment problem with offsets. In 35th Annual Symposium on Combinatorial Pattern Matching (CPM 2024), pages 26–1. Schloss Dagstuhl–Leibniz-Zentrum für Informatik, 2024.

“고등부 4번. 축제” 문제 풀이

작성자: 이종영

부분문제 1

$dp[u][d]$ 를 u 를 루트로 한 서브트리에서, u 에서 리프로 가는 경로들에서 선택된 도로의 수가 d 이하일 때, 일일 이용량 합의 최댓값이라고 정의하자. 답은 각 정점 u 에 대해 $dp[u][K]$ 이다.

일일 이용량이 w 인 도로를 추가하는 경우 전이는 도로를 선택하지 않은 경우 $dp[d] \rightarrow dp'[d]$ 와 도로를 선택한 경우 $dp[d] + w \rightarrow dp'[d+1]$ 이다.

$dp[u]$ 와 $dp[v]$ 를 합치는 경우 전이는 $d_1 + d_2 \leq K$ 인 d_1, d_2 에 대해 $dp[u][d_1] + dp[v][d_2] \rightarrow dp'[\max(d_1, d_2)]$ 이다.

이대로 구현할 경우 시간복잡도가 $O(NK^2)$ 이지만, 다양한 방법으로 $O(NK)$ 로 최적화할 수 있다. $\max(d_1, d_2)$ 를 고정하는 방법으로 $O(K)$ 에 합치거나, d 의 값이 서브트리의 높이 이하임을 이용해 $O(\min(\text{depth}(u), K) \cdot \min(\text{depth}(v), K))$ 에 합치는 방법 등이 있다.

부분문제 5

모든 일일 이용량이 1로 동일하다. 이 경우 하나의 트리에 대해서 리프와 연결된 도로들을 선택한 후 지우는 것을 $\lfloor \frac{K}{2} \rfloor$ 번 반복하고, K 가 홀수이고 아직 남은 도로가 존재한다면 추가로 하나 사용하는 것이 최적이다.

모든 서브트리에 대해 답을 구하는 경우, 각 도로가 해당 도로의 위 정점에서 루트로 향하다가 멈추는 어떠한 경로 상의 정점들의 답에 기여한다는 사실을 이용하면 $O(N \log N)$ 에 해결할 수 있다.

부분문제 9

각 dp 배열들이 d 에 대해 오목함을 증명한다.

일일 이용량이 w 인 도로를 추가하는 경우 민코프스키 합과 같은 원리로 오목성이 유지된다.

$dp[u]$ 와 $dp[v]$ 를 합치는 경우, 오목하고 단조증가하는 두 배열 A 와 B 가 있을 때, $C[k] = \max_{i \leq k, j \leq k, i+j \leq K} A[i] + B[j]$ 로 정의되는 배열 C 가 오목함을 증명하면 된다. $k \leq \frac{K}{2}$ 의 경우 $C[k] = A[k] + B[k]$ 이니 자명하다. $k > \frac{K}{2}$ 의 경우 $C[k] = \max_{K-k \leq i \leq k} A[i] + B[K-i]$ 이다. 특정 k 에 대한 최적 i 를 $f(k)$ 로 정의할 시, $K-k \leq \lfloor \frac{f(k-1)+f(k+1)}{2} \rfloor \leq \lceil \frac{f(k-1)+f(k+1)}{2} \rceil \leq k$ 임을 이용해 $2C[k] \geq C[k-1] + C[k+1]$ 임을 증명할 수 있다.

이제 dp 배열들을 $dp[d] - dp[d-1]$ 값들의 multiset으로 표현할 수 있다.

일일 이용량이 w 인 도로를 추가하는 경우의 전이는 단순히 multiset에 w 를 추가하고, 크기가 K 보다 크다면 가장 작은 값을 빼내는 것으로 충분하다.

$h_1 \leq h_2$ 에 대해 높이가 h_1, h_2 인 두 dp 배열을 $O(h_1 \log K)$ 에 합칠 수 있다면 전체 문제를 $O(N \log K)$ 에 해결할 수 있다. $h_1 \geq \frac{K}{2}$ 인 경우 부분문제 1과 같이 $O(K)$ 등에 합쳐주면 된다. 그렇지 않은 경우, $\max(d_1, d_2) = d_1$ 또는 $d_1 + d_2 > K$ 가 가능한 d_2 는 앞과 뒤의 길이 h_1 부분임을 알 수 있다. 이 부분에 해당되는 dp 값만 직접 구하면, 나머지 값들은 평행이동되기에 차이가 변하지 않음을 이용하면 $O(h_1 \log K)$ 에 처리가 가능하다.